Algorithmes et Pensée Computationnelle

Algorithmes de tri et Complexité - exercices basiques

Le but de cette série d'exercices est d'aborder les notions présentées durant la séance de cours. Cette série d'exercices sera orientée autour des points suivants :

1. La récursivité
2. La complexité des algorithmes
3. Les algorithmes de tri

Les exercices de cette séance seront réalisés en Java et Python.

Le temps indiqué (🕓) est à titre indicatif.

1 Récursivité

Le but principal de la récursivité est de résoudre un gros problème en le divisant en plusieurs petites parties à résoudre.

Question 1: (🕓 *10 minutes*) **Somme des chiffres**
Écrivez une fonction récursive en Python et en Java qui prend un entier en paramètre et retourne la somme des chiffres dont il est composé. Par exemple, la somme des chiffres de 126 est : 1+2+6 = 9.

> **Conseil**
>
> Pour obtenir le chiffre des unités d'un nombre, utilisez l'opérateur modulo `%`. Par exemple : $42 \bmod 10 = 2$, car le reste de la division entière de 42 par 10 est 2. Ensuite, pour obtenir le chiffre 4, effectuez une division entière par 10 : en Python, `42 // 10` vaut 4, en Java, `42 / 10` vaut 4, car les deux opérandes sont des entiers.

2 Complexité

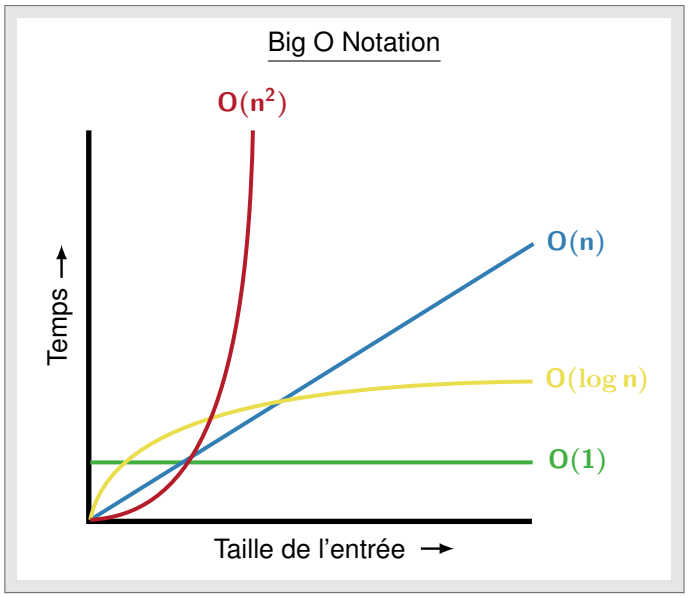


FIGURE 1 – Représentation de complexités temporelles. La complexité décrit la façon dont le temps d'exécution **croît asymptotiquement** lorsque la taille des données augmente, et non sa valeur exacte.

Indiquez en une phrase, ce que font ces algorithmes ci-dessous et calculez leur complexité temporelle avec la notation O. Le code est écrit en Python et en Java.

Question 2: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?
Python :

```python
# Entrée: n un nombre entier
def algo(n):
    s = 0
    for i in range(10*n):
        s += i
    return s
```

Java :

```java
public static int algo(int n) {
    int s = 0;
    for (int i=0; i < 10*n; i++){
        s += i;
    }
    return s;
}
```

(A) $O(n)$
(B) $O(n^3)$
(C) $O(\log(n))$
(D) $O(n^n)$

Conseil

Rappelez-vous que la notation O sert à exprimer la complexité d'algorithmes dans le **pire des cas**. Les règles suivantes vous seront utiles. Soit n la taille de vos données. On a alors :

1. Les constantes sont ignorées : $O(2n) = 2 \cdot O(n) = O(n)$
2. Les termes dominés sont ignorés : $O(2n^2 + 5n + 50) = O(n^2)$

Question 3: (*10 minutes*) **Complexité**
Soit `values` une liste de longueur n. Quelle est la complexité des programmes ci-dessous ?
Python :

```python
# Entrée: values est une liste de nombres entiers et threshold un nombre entier
def algo(values, threshold):
    i = 0
    while i < len(values) and values[i] <= threshold:
        i += 1
    s = i - 1
    return s

```

Java :

```java
public static int algo(int[] values, int threshold) {
    int i = 0;
    while (i < values.length && values[i] <= threshold){
        i += 1;
    }
    int s = i - 1;
    return s;
}

```

(A) $O(n^3)$

(B) $O(\log(n))$

(C) $O(n)$

(D) $O(n^n)$

Question 4: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?

Python :

```python
# Entrée: first_values et second_values sont deux listes de nombres entiers
def algo(first_values, second_values):
    n = len(first_values)
    m = len(second_values)
    for i in range(n):
        first_values[i] = first_values[i]*2
    for j in range(m):
        second_values[j] = second_values[j]%2

```

Java :

```java
public static void algo(int[] firstValues, int[] secondValues) {
    int n = firstValues.length;
    int m = secondValues.length;
    for (int i=0; i < n; i++){
        firstValues[i] = firstValues[i]*2;
    }
    for (int j=0; j < m; j++){
        secondValues[j] = secondValues[j]%2;
    }
}

```

(A) $O(n^2)$

(B) $O(n+m)$

(C) $O(n)$

(D) $O(2^n)$

Question 5: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?

Python :

```python
# Entrée: values est une liste de nombres entiers
import math
def algo(values):
    n = len(values)
    i = 1
    s = 0
    while i < n:
        s += values[i]
        i = 2*i
    return s

```

Java :

```java
import java.lang.Math;

public static int algo(int[] values) {
    int n = values.length;
    int i = 1;
    int s = 0;
```

```
    while (i < n){
        s += values[i];
        i = 2*i;
    }
    return s;
}
```

(A) $O(n^2)$
(B) $O(n)$
(C) $O(\log(n))$
(D) $O(n^n)$

Question 6: (*10 minutes*) **Complexité** Optionnel
Quelle est la complexité du programme ci-dessous ?
Python :

```
# Entrée: n un nombre entier
def algo(n):
    m = 0
    for i in range(n):
        for j in range(i):
            m += i+j
    return m

```

Java :

```
public static int algo(int n) {
    int m = 0;
    for (int i=0; i < n; i++){
        for (int j=0; j < i; j++){
            m += i+j;
        }
    }
    return m;
}

```

(A) $O(n^2)$
(B) $O(n)$
(C) $O(\log(n))$
(D) $O(2^n)$

3 Algorithmes de Tri

Question 7: (*15 minutes*) [Python] **Insertion dans une liste triée**
Soit un nombre entier `n`, et une liste triée `l`. Ecrivez un programme Python qui insère la valeur `n` dans la liste `l` tout en s'assurant que la liste `l` reste triée.

```
def insertion_entier(liste, number):
    # TODO : Compléter ici

print(insertion_entier([2, 4, 6], 1))
```

Exemple

En passant les arguments suivants à votre programme : `n=5` et `l=[2,4,6]`. Ce dernier devra retourner `l =[2,4,5,6]`

Question 8: (*30 minutes*) [Python] **Tri fusion (Merge Sort)**
Étant donné deux listes triées, on peut construire une liste triée contenant tous leurs éléments (leur *fusion*). Elle repose sur l'observation que si les deux listes sont triées, le plus petit élément de la liste fusionnée est soit le premier élément de la première liste, soit le premier élément de la seconde. On construit donc la liste finale élément par élément : à chaque étape, on compare les premiers éléments des deux listes, on retire le plus petit et on l'ajoute à la fin de liste finale. Dès que l'une des deux listes est vide, il suffit d'ajouter à la suite les éléments restants de l'autre.

Les étapes à suivre pour implémenter l'algorithme sont les suivantes :

1. Si le tableau n'a qu'un élément, il est déjà trié.
2. Sinon, séparer le tableau en deux parties plus ou moins égales.
3. Trier récursivement les deux parties avec l'algorithme de tri fusion.
4. Fusionner les deux tableaux triés en un seul tableau trié.

Soit la liste `l` suivante [38, 27, 43, 3, 9, 82, 10], triez les éléments de la liste en utilisant un tri fusion. Combien d'itération effectuez-vous ?

- **Python :**

```python
def merge(partie_gauche, partie_droite):
    # TODO: Code à compléter

def tri_fusion(liste):
    # TODO: Code à compléter

if __name__ == "__main__":
    l = [38, 27, 43, 3, 9, 82, 10]
    print(tri_fusion(l))
```

Conseil

- L'algorithme est récursif.
- Revenez à la visualisation de l'algorithme dans les diapositives 81 à 109 pour comprendre comment fonctionne concrètement le tri fusion.

Conseil

Théorème 1 La récurrence

$$T(n) = a\,T(n/b) + cn^k$$
$$T(1) = c$$

où a, b, c et k sont des constantes, se résout comme suit :

$$T(n) \in O(n^k) \quad \text{si } a < b^k$$
$$T(n) \in O(n^k \log n) \quad \text{si } a = b^k$$
$$T(n) \in O(n^{\log_b a}) \quad \text{si } a > b^k$$

Question 9: (*20 minutes*) **Tri à bulles (Bubble Sort) - Python** Optionnel
Le tri à bulles consiste à parcourir une liste et à comparer ses éléments. Le tri est effectué en permutant les éléments de telle sorte que les éléments les plus grands soient placés à la fin de la liste.
Concrètement, si un premier nombre x est plus grand qu'un deuxième nombre y et que l'on souhaite trier l'ensemble par ordre croissant, alors x et y sont mal placés et il faut les inverser. Si, au contraire, x est plus petit que y, alors on ne fait rien et l'on compare y à z, l'élément suivant.
Soit la liste `l = [1, 2, 4, 3, 1]`, triez les éléments de la liste en utilisant un tri à bulles. Combien d'itérations effectuez-vous ?

- **Python :**

```python
def tri_bulle(l):
    for i in range(len(l)):
        #TODO: Code à compléter

if __name__ == "__main__":
    l = [1, 2, 4, 3, 1]
    tri_bulle(l)
    print(l)
```